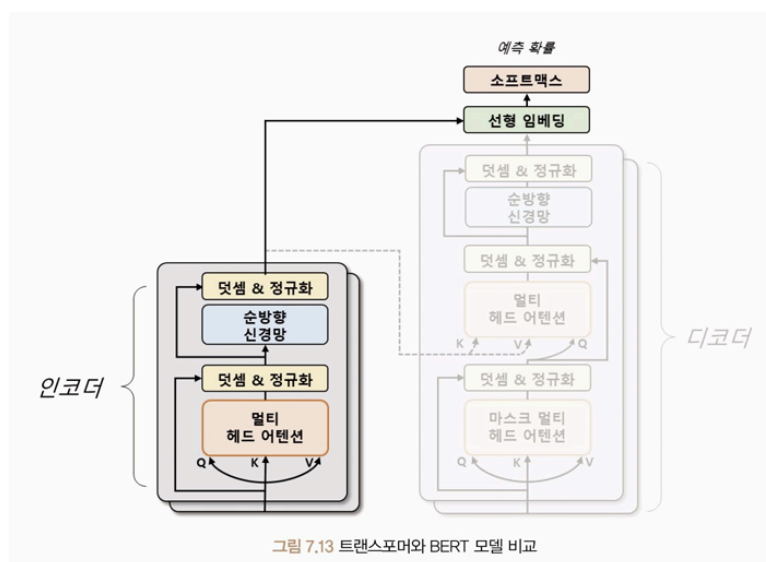


7장 트랜스포머 (BERT~)

BERT (Bidirectional Encoder Representations from Transformers)

- 2018년 구글 발표 트랜스포머 기반 양방향 인코더 언어 모델.
- 양방향 인코딩: 입력 시퀀스를 이전·이후 단어 모두를 참조하여 단어 의미와 문맥을 동시에 파악. 기존 단방향 모델의 한계(이전 단어만 고려)를 보완.
- 대규모 말뭉치 사전 학습 후 전이 학습에 주로 활용. 적은 레이블 데이터로도 높은 정확도를 달성하여 학습 시간 감소.

그림 7.13 트랜스포머와 BERT 모델 비교



- 인코더만 사용하여 입력 문장을 처리(디코더는 사용하지 않음).
- 트랜스포머 인코더 모듈을 여러 블록 쌓아 구성.

사전 학습 방법

- BERT는 다음 두 가지 사전 학습 과제를 사용:
 1. **MLM (Masked Language Modeling)**: 문장 일부 단어를 **[MASK]**로 가리고 해당 단어 예측.

- 예: "I'm learning PyTorch" 에서 "learning" 을 가리면 "I'm [MASK] PyTorch" , BERT는 [MASK] 위치의 단어를 예측.

2. **NSP (Next Sentence Prediction)**: 두 문장이 주어졌을 때 두 번째 문장이 첫 문장의 다음 문장인지 여부 판별.

- 두 문장 간 연속성/의미적 유사성 학습.

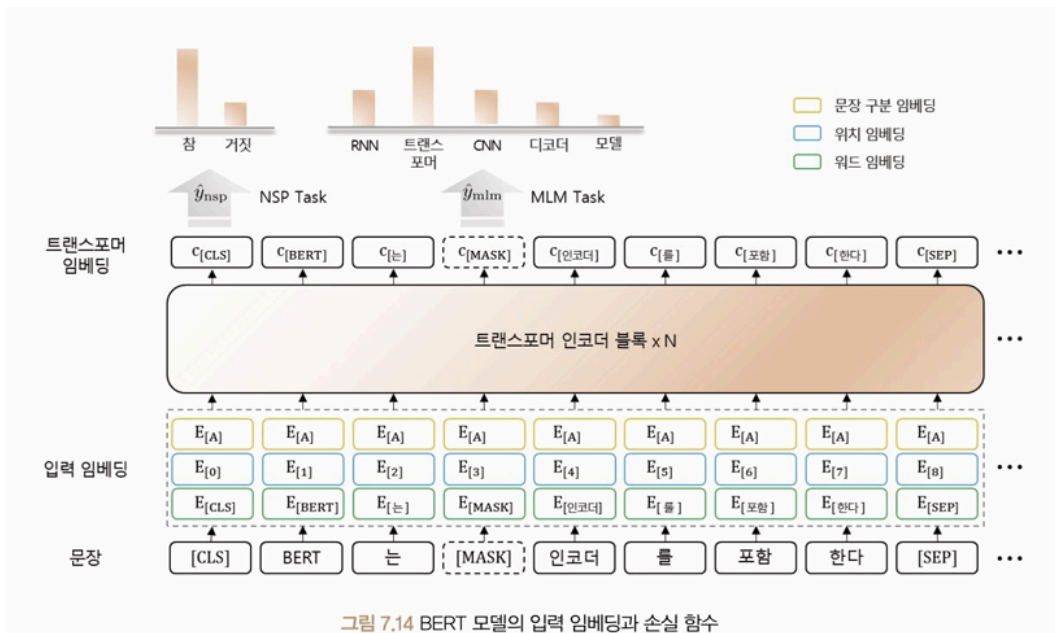
특수 토큰

- [CLS]**: 입력 문장 시작에 추가. 문장 분류 시 대표 벡터로 사용(공·부정 등).
- [SEP]**: 하나의 입력 내 두 문장 이상을 구분.
- [MASK]**: MLM 학습에서 가릴 단어 위치를 표시하고 예측에 활용.

BERT 입력 시퀀스 형태

- 예시 구조: [CLS] 문장-1 [SEP] 문장-2 [MASK] [SEP]

그림 7.14 BERT 입력 임베딩과 손실 함수



- 입력 임베딩은 토큰 임베딩 + 위치 임베딩 + 문장 구분 임베딩의 합.
- 상단에 **NSP Task**, 하단에 **MLM Task**가 동시에 학습됨.

[MASK] 적용 비율

- 전체 토큰의 **15%**를 마스킹:

- 그중 **80%**는 **[MASK]**로 대체,
- **10%**는 임의의 단어로 변경,
- **10%**는 원래 단어 유지.

모델 실습

예제 7.15 네이버 영화 리뷰 데이터 불러오기

```
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
df = pd.DataFrame(corpus.test).sample(20000, random_state=42)
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(train.head(5).to_markdown())
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

데이터 분할 결과

- **Training:** 12,000
- **Validation:** 4,000
- **Testing:** 4,000

예제 7.16 BERT 입력 텐서 생성

```
import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
```

```

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

```

토크나이저 및 데이터로더 준비

```

epochs = 5
batch_size = 32
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    do_lower_case=False
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)

```

```
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])
```

- 설명:
 - **bert-base-multilingual-cased**: 다국어, 대소문자 유지.
 - **RandomSampler**: 학습용, 무작위 샘플링.
 - **SequentialSampler**: 검증·평가용, 고정 순서.

예제 7.17 BERT 모델 선언

```
from torch import optim
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

- **BertForSequenceClassification**으로 문장 분류 모델 로드.
- **AdamW(학습률 1e-5, eps 1e-8)** 사용.

BERT 모델 구조 출력 결과

```
bert
├─ embeddings
│   ├─ word_embeddings
│   ├─ position_embeddings
│   └─ token_type_embeddings
├─ LayerNorm
├─ dropout
└─ encoder
```

```
|   └─ layer (L0 ... L11)
|   └─ pooler
|   └─ dense
|   └─ activation
|   └─ dropout
└─ classifier
```

BERT 입력 임베딩·인코더·분류기

- 입력 임베딩: `word_embeddings` + `position_embeddings` + `token_type_embeddings` → **LayerNorm·Dropout** 적용.
- 인코더 블록(총 12개): 셀프 어텐션 + FFN으로 다양한 관계 학습.
- 풀러(pooler): `[CLS]` 벡터에 선형 → Tanh. 이후 **Dropout**.
- 분류기(classifier): 최종적으로 `[CLS]` 벡터로 결과 예측.

학습 및 평가 결과

BERT 모델 학습 결과

Epoch 1: Train Loss: 0.5744 Val Loss: 0.4519 Val Accuracy 0.7867
Saved the model weights
Epoch 2: Train Loss: 0.4224 Val Loss: 0.4503 Val Accuracy 0.7957
Saved the model weights
Epoch 3: Train Loss: 0.3320 Val Loss: 0.4207 Val Accuracy 0.8147
Saved the model weights

BERT 모델 평가 결과

Test Loss : 0.4183
Test Accuracy : 0.8085

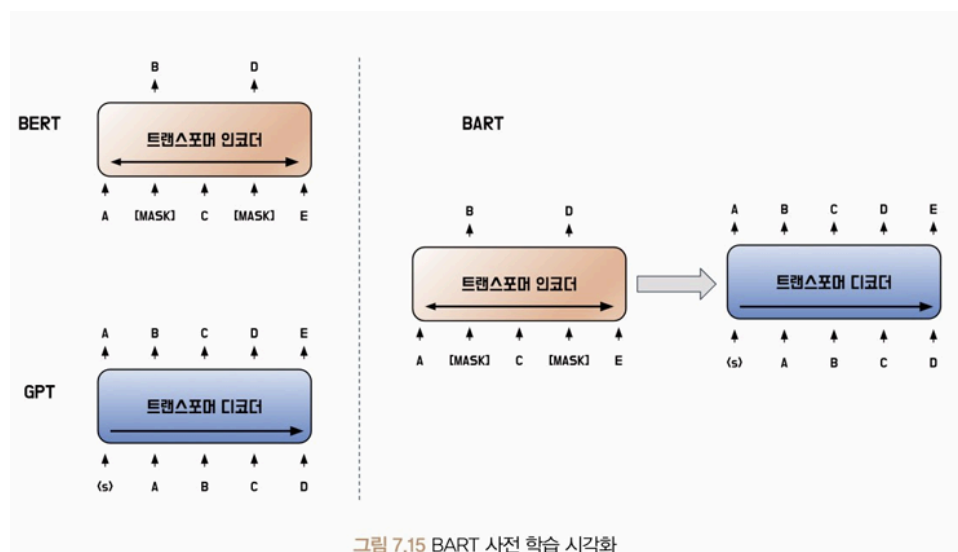
- 설명: 에폭이 진행될수록 학습 손실 감소, 검증 정확도 증가 → 과대적합 없이 적절 학습으로 해석.

BART

BART 소개

- **BART (Bidirectional Auto-Regressive Transformer)**: 2019년 메타(FAIR) 발표, 트랜스포머 기반 모델.
- 구조: BERT의 인코더 + GPT의 디코더를 결합한 Seq2Seq(시퀀스-투-시퀀스).
- 사전 학습 방식: 노이즈 제거 오토인코더(Denoising Autoencoder)
 - 입력 텍스트에 의도적으로 잡음(노이즈)을 추가 → 원본 문장 복원을 학습.
- 비교 요약
 - **BERT 인코더**: 일부 단어를 **마스킹**하여 문맥 전체를 이해(양방향 문맥·상호작용 파악).
 - **GPT 디코더**: 이전 토큰들을 입력받아 다음 토큰을 예측(순차 생성 능력).
 - **BART**: 두 방식을 일반화하여, 양방향 문맥 정보와 생성 능력을 함께 갖추.

그림 7.15 — BART 사전 학습 시각화



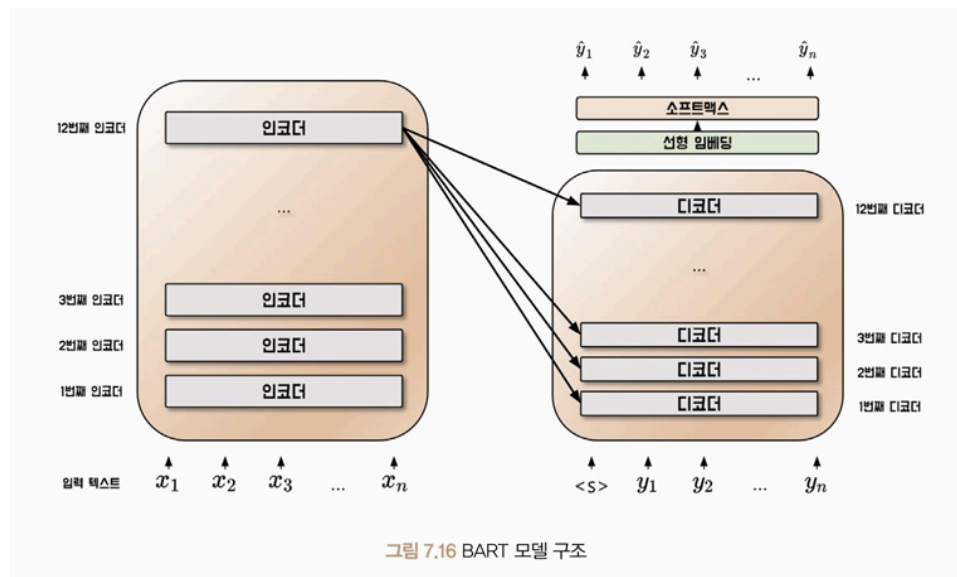
- 왼쪽: BERT(인코더) — 마스킹된 입력으로 문맥을 복원.

- 가운데: **BART(인코더→디코더)** — 노이즈가 섞인 입력을 인코더에 넣고, 디코더가 원문 재생성.
- 아래: GPT(디코더) — 오른쪽으로 순차 생성.

효과: 인코더만으로는 한계가 있던 생성을 디코더로 보완, 반대로 디코더만으로는 약했던 양방향 문맥 반영을 인코더로 보완.

BART 모델 구조

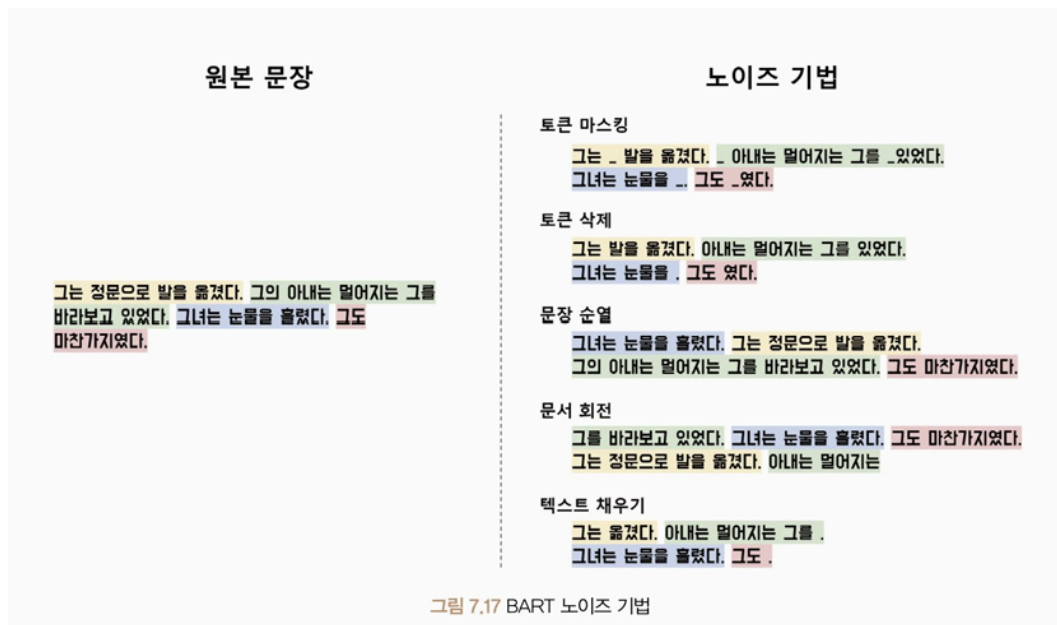
그림 7.16 — BART 모델 구조



- 인코더/디코더 모두 사용하는 트랜스포머 유사 구조.
- 차이점: 트랜스포머(원형)는 인코더 모든 계층 ↔ 디코더 모든 계층 사이에 어텐션을 수행하지만,
BART는 인코더의 마지막 계층 ↔ 디코더 각 계층 사이에서만 인코더-디코더 어텐션을 수행.
- 동작 개요
 - 인코더: 각 단어 임베딩 → 여러 층을 거쳐 **문장 전체 의미를 대표**하는 벡터들을 산출.
 - 디코더: 이전 시점 출력과 인코더 표현을 **어텐션**으로 참조하여 **출력 문장 생성**.

사전 학습 방법 — 노이즈 기법

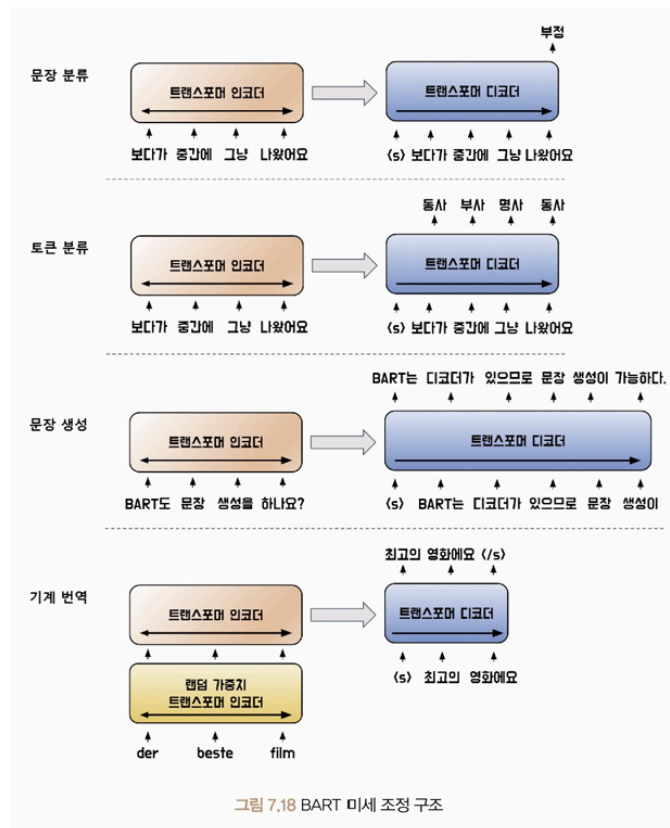
그림 7.17 — BART 노이즈 기법



- **토큰 마스크(Token Masking)**: 일부 토큰을 **[MASK]**로 치환(MLM과 동일 개념).
- **토큰 삭제(Token Deletion)**: 임의 토큰 삭제(삭제 위치 정보도 모델이 파악해야 함).
- **문장 순열(Sentence Permutation)**: 문단을 문장 단위로 섞기(문장 연결 구조 학습).
- **문서 회전(Document Rotation)**: 임의 토큰을 문서 시작으로 만들어 **시작점 변화**에 적응.
- **텍스트 채우기(Text Infilling)**: 여러 토큰을 하나의 **구간(span)**으로 묶어 **구간 마스크**
 - 구간 길이는 확률적(예: 포아송 분포의 매개변수 값을 설정하여 결정).
 - 모델은 연속된 마스크를 복구하고, 실제로 마스크되지 않은 토큰도 **구분**할 수 있어 **문장 이해력 향상**.

미세 조정(파인튜닝)

그림 7.18 — BART 미세 조정 구조



- **문장 분류:** 입력 문장을 인코더·디코더 모두에 동일 입력 → 디코더 마지막 토큰 은닉 상태를 선형 분류기 입력으로 사용(전체 입력과 어텐션이 반영된 은닉 상태 활용).
- **토큰 분류:** 입력 동일 → 디코더 각 시점의 은닉 상태를 토큰 분류 입력으로 사용(시점별 은닉 + 어텐션).
- **문장 생성:** 디코더가 있으므로 생성 작업 수행 가능.
- **기계 번역:** 입력/출력이 모두 텍스트인 작업에 적합.

적합 작업

- 추상적 질의응답(**Abstractive QA**), 문장 요약(**Summarization**) 등 출력 생성이 필요한 과업에 효과적.

학습 절차(2단계)

1. 새로 추가된 인코더/입력 임베딩/일부 셀프 어텐션 가중치 등 **일부만 우선 학습**.
2. 이후 **모든 가중치를 작은 학습률로 전체 미세 조정**.

모델 실습

데이터셋 준비 (뉴스 요약)

- 사용 데이터: **Argilla**가 공개한 **뉴스 요약** 데이터(뉴스 본문 + 짧은 요약 텍스트).
- Hugging Face **datasets** 라이브러리로 불러옴.

```
# 데이터셋 라이브러리 설치
pip install datasets
```

예제 7.18 — 뉴스 요약 데이터셋 불러오기

```
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])

train, valid, test = np.split(
    df.sample(frac=1, random_state=42),
    [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(f"Source News : {train.text.iloc[0][:200]}")
print(f"Summarization : {train.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

- **출력 예:** 원문(일부), 요약(일부)과 함께 **Train 3000 / Val 1000 / Test 1000**.

BART 입력 텐서 생성

예제 7.19 — BART 입력 텐서 생성

```
import torch
from transformers import BartTokenizer
```

```

from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    labels = []
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)

    for target in data.prediction:
        labels.append(tokenizer.encode(target, return_tensors="pt").squeeze())
    labels = pad_sequence(labels, batch_first=True, padding_value=-100).to(
device)

    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

```

- **요약 작업 특징:** 입력·출력 모두 텍스트 시퀀스 → 레이블(labels)은 요약 텍스트의 토큰 ID 시퀀스.
- **길이 맞춤:** 시퀀스 길이가 서로 다르므로 패딩 적용.
- **손실에서 패딩 무시:** 패딩 값으로 100 사용(손실 계산 시 무시되도록 설정).

토크나이저/데이터로더 구성 & 확인

```

epochs = 3
batch_size = 8
device = "cuda" if torch.cuda.is_available() else "cpu"

tokenizer = BartTokenizer.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])

```

- **토크나이저:** `facebook/bart-base` (사전 학습된 BART 기본 버전).
- **샘플 출력:** `(input_ids, attention_mask, labels)` 텐서 3개가 출력되며, `labels`에는 패딩이 `-100`으로 채워져 표시됨.

정리

- **BART = 인코더(양방향 문맥) + 디코더(생성)**를 결합한 **노이즈 제거형 사전 학습 모델**.
- 다양한 **노이즈 기법**(마스킹/삭제/순열/회전/채우기)으로 **강인한 복원·이해 능력**을 학습.
- **요약·생성·번역** 등 **출력 생성 과업**에 특히 적합.
- 요약 학습에서 **정답 시퀀스(요약)**를 `labels`로 두고, **손실 계산 시 패딩은 -100으로 무시**.

예제 7.20 BART 모델 선언

```

from torch import optim
from transformers import BartForConditionalGeneration

model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
).to(device)

optimizer = optim.AdamW(model.parameters(), lr=5e-5, eps=1e-8)

```

- **BartForConditionalGeneration**: 입력 시퀀스로부터 **출력 시퀀스**를 생성하는 BART 변형. (문장 요약, 기계 번역, 질의응답 등)
- **버전**: `facebook/bart-base` (인코더/디코더 **각 6개** 계층).
`facebook/bart-large` 는 **각 12개** 계층.
- **최적화**: `AdamW` , 학습률 `5e-5` , `eps=1e-8` .

BART 모델 구조 출력 결과

```

model
├─ shared
├─ encoder
│   ├── embed_tokens
│   ├── embed_positions
│   └─ layers (L0 ... L5)
│       └─ layernorm_embedding
├─ decoder
│   ├── embed_tokens
│   ├── embed_positions
│   └─ layers (L0 ... L5)
│       └─ layernorm_embedding
└─ lm_head

```

- **shared**: 인코더·디코더가 **임베딩 계층**을 공유.
- **인코더/디코더**: 각 **6개** 계층.
- **layernorm_embedding**: 임베딩 마지막 단계에서 정규화.

- **출력:** 디코더 마지막 계층의 은닉 상태 → **단어 사전 크기**의 완전연결 계층을 거쳐 언어 모델 출력을 형성.

문장 생성 평가 — ROUGE

- 사용 지표: **ROUGE-N**(N-gram 재현율/정밀도), **ROUGE-L**, **ROUGE-LSUM**, **ROUGE-W**.
- 그림 7.19:

정답 문장 유니그램	[대한민국, 은, 16, 강, 에, 진출, 했다]
생성된 문장 유니그램	[대한민국, 은, 8, 강, 에, 진출, 하지, 못, 했다]
ROUGE-1 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$
ROUGE-1 정밀도	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$
정답 문장 바이그램	[대한민국은, 은 16, 16강, 강에, 에 진출, 진출했다]
생성된 문장 바이그램	[대한민국은, 은 8, 8강, 강에, 에 진출, 진출하지, 하지 못, 못 했다]
ROUGE-2 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$

그림 7.19 루지 점수 계산 방법

- **ROUGE-1 재현율** = (정답 문장과 생성 문장에 **공통 유니그램** 개수) / (정답 문장 유니그램 개수)
- **ROUGE-1 정밀도** = (공통 유니그램 개수) / (생성 문장 유니그램 개수)
- **ROUGE-2 재현율**은 **바이그램** 기준.
- **ROUGE-L**: 정답/생성 간 **최장 공통 부분 수열(LCS)** 기반.
- **ROUGE-LSUM**: 문장 경계를 인식하여 **union-LCS** 등으로 중복을 보정.
- **ROUGE-W**: **가중 LCS**(연속 구간에 가중치 부여).

평가 라이브러리 설치

```
pip install evaluate rouge_score absl-py
```

예제 7.21 BART 모델 학습 및 평가

ROUGE 계산 함수

```
import numpy as np
import evaluate

def calc_rouge(preds, labels):
    preds = preds.argmax(axis=-1)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    rouge2 = rouge_score.compute(
        predictions=decoded_preds,
        references=decoded_labels
    )
    return rouge2["rouge2"]

rouge_score = evaluate.load("rouge", tokenizer=tokenizer)
```

- `labels`의 패딩 `-100`은 패드 토큰으로 치환 후 디코딩.
- 반환값은 **ROUGE-2 점수**.

검증 루프

```
def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss, val_rouge = 0.0, 0.0
```



```

for input_ids, attention_mask, labels in dataloader:
    outputs = model(input_ids=input_ids,
                    attention_mask=attention_mask,
                    labels=labels)
    logits = outputs.logits
    loss = outputs.loss

    rouge = calc_rouge(logits.detach().cpu().numpy(),
                      labels.to("cpu").numpy())

    val_loss += loss
    val_rouge += rouge

val_loss /= len(dataloader)
val_rouge /= len(dataloader)
return val_loss, val_rouge

```

학습 로그

```

Epoch 1: Train Loss: 2.1677 Val Loss: 1.8385 Val Rouge 0.2564
Epoch 2: Train Loss: 1.6036 Val Loss: 1.9068 Val Rouge 0.2608
Epoch 3: Train Loss: 1.2503 Val Loss: 1.9990 Val Rouge 0.2538
Saved the model weights

```

rouge2 예시 출력

```

{
  'rouge1': 0.4411699...,
  'rouge2': 0.2,
  'rougeL': 0.4275670...,
  'rougeLsum':0.4243807...
}

```

예제 7.22 BART 모델 평가

```
model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
).to(device)
model.load_state_dict(torch.load("../models/BartForConditionalGeneration.
pt"))

test_loss, test_rouge_score = evaluation(model, test_dataloader)
print(f"Test Loss : {test_loss:.4f}")
print(f"Test ROUGE-2 Score : {test_rouge_score:.4f}")
```

출력

```
Test Loss : 1.7890
Test ROUGE-2 Score : 0.2699
```

- **ROUGE-2**는 0~1 사이. **소규모 샘플링**으로 학습한 점을 고려하면 **중간 수준**의 성능.

예제 7.23 문장 요약문 비교

```
from transformers import pipeline

summarizer = pipeline(
    task="summarization",
    model=model,
    tokenizer=tokenizer,
    max_length=54,
    device="cpu"
)

for index in range(5):
    news_text = test.text.iloc[index]
    summarization = test.prediction.iloc[index][0]["summary_text"]
    predicted_summarization = summarizer(news_text)[0]["summary_text"]
    print(f"정답 요약문 : {summarization}")
```

```
print(f"모델 요약문 : {predicted_summarization}\n")
```

- 출력: 정답 요약문 vs 모델 요약문을 문장별로 직접 비교.

ELECTRA

개요

- ELECTRA**(Efficiently Learning an Encoder that Classifies Token Replacements Accurately): 2020년 구글 발표 **트랜스포머 기반** 모델.
- 접근: **생성자(Generator) + 판별자(Discriminator)**를 함께 학습.
 - Generator**: MLM 방식으로 **마스크된 토큰을 생성**.
 - Discriminator**: 각 토큰이 **원본인지/생성자로 치환된 토큰인지 판별(RTD: Replaced Token Detection)**.

사전 학습 방법

- 두 모델 모두 **트랜스포머 인코더** 구조.
- 그림 7.20**:
 - 입력 문장의 **15%**를 마스크 → **생성자**가 해당 위치 토큰을 예측/대체.
 - 판별자**는 전체 시퀀스를 입력받아 **각 토큰이 바뀌었는지 여부**를 분류.

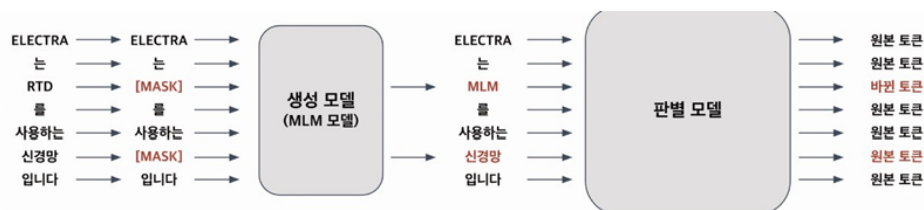


그림 7.20 ELECTRA 모델 학습 방법

- GAN과의 차이**
 - 생성자는 **무작위 노이즈** 대신 **마스크 처리된 텍스트**를 입력으로 사용.
 - 생성자가 **원본과 동일한 토큰**을 내도 ELECTRA에서는 **원본으로 간주**.
 - 판별자**는 실제/가짜를 **토큰 단위로 학습**.
- 효율화**

- 생성자 성능이 과도하게 높아지면 판별자 학습이 어려움 → **생성자 크기를 판별자의 1/2~1/4로 설정.**
- **모든 가중치 공유 대신 임베딩 계층 가중치만 공유.**
- 사전 학습 후 **다운스트림에서는 판별자만 사용.**

모델 실습

- 데이터: **네이버 영화 리뷰 감정 분석**(BERT와 동일 데이터로 비교).
- 토큰나이저 불러오기(전처리 흐름은 BERT와 동일하게 일부 변경).

```
import torch
from transformers import ElectraTokenizer
# 이후 예제 7.16의 전처리 코드를 ELECTRA에 맞게 일부 수정하여 사용
```

1) 토큰나이저 불러오기

```
from transformers import ElectraTokenizer

tokenizer = ElectraTokenizer.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discrimi-
nator",
    do_lower_case=False,
)
```

- 출력 예: `input_ids`, `attention_mask` 등이 텐서로 생성됨.

모델 선택 메모

- 영어: `google/electra-small` (14M), `google/electra-base` (110M), `google/electra-large` (330M)
- 한국어: `monologg/koelectra-small-v3` (14M), `monologg/koelectra-base-v3` (110M)
- 다운스트림 분류용은 **discriminator** 가중치 사용:

`monologg/koelectra-base-discriminator` (생성기 사용 시 `...-generator`)

2) 분류 모델 선언 & 옵티마이저

```

from torch import optim
from transformers import ElectraForSequenceClassification

model = ElectraForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    num_labels=2
).to(device)

optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)

```

모델 구조

- **embeddings** : word/position/token_type + LayerNorm + dropout
 - **encoder** : 12개 계층(L0 ... L11)
 - **classifier** : dense
- BERT와 유사한 인코더 스택 + CLS 기반 분류 헤드

3) 학습/평가 결과

Validation

- Epoch 1: Train 0.4489 / Val 0.3227 / **Val Acc 0.8715**
- Epoch 2: Train 0.2891 / Val 0.2928 / **Val Acc 0.8808**
- Epoch 3: Train 0.2188 / Val 0.3111 / **Val Acc 0.8802**

Test

- **Test Loss 0.3084 / Test Accuracy 0.8732**

■ 비교: 교재 기준 BERT(80.85%) 대비 +약 7.5%p 정확도 향상

4) 벤치마크 메모

- ELECTRA는 GLUE에서 높은 점수 기록
 - **electra-small** 이 **bert-small** 대비 **+5%p** 내외
 - **electra-base** vs **bert-base** **+~3%p**, **electra-large** vs **bert-large** **+~2%p**

- 효율성: 12시간 학습 electra-small > 4일 학습 bert-small (성능/시간 모두 유리)

T5

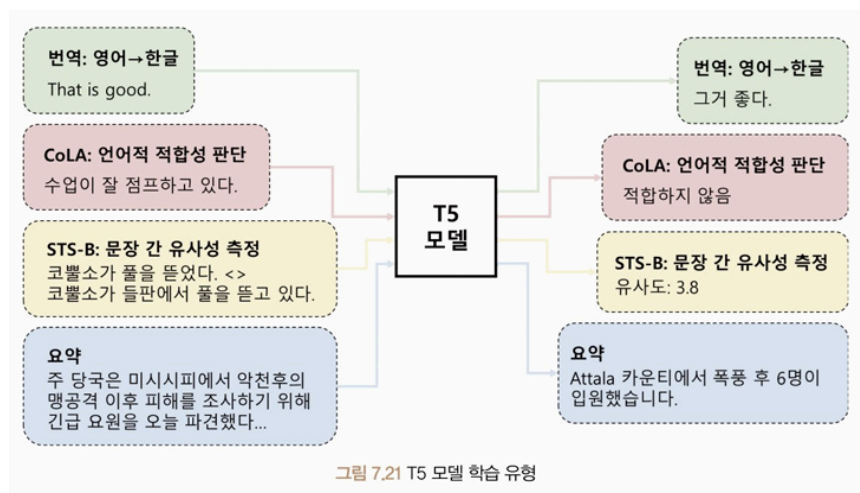
1) 개요

- T5(Text-to-Text Transfer Transformer, 2019)
- 인코더-디코더 구조, 모든 작업을 텍스트→텍스트로 통일
- GLUE, SuperGLUE, CNN/DailyMail 요약 등에서 SOTA 달성

활용 작업 예시

- 번역, 요약, 질의응답, 텍스트 분류, 의미 유사도(STS-B), 문법성(CoLA) 등
- 프롬프트로 작업 지정: "translate English to German:", "summarize:" 등

그림 7.21 - T5 모델 학습 유형 요약



T5는 텍스트→텍스트 프레임으로 다양한 과제를 같은 방식으로 처리한다:

- **번역 (영↔한)**
입력: 영어 문장 → 출력: 한국어 번역(예: "That is good." → "그거 좋다.")
- **문법 적합성 판별 (CoLA)**
문장이 문법적으로 **적합/부적합** 인지 라벨 생성.
- **문장 의미 유사도 (STS-B)**
두 문장 쌍의 **유사도 점수**(연속값) 예측.

- **요약 (Summarization)**

긴 본문을 **간결한 요약문**으로 변환.

→ 동일한 T5에 **과제별 프롬프트/형식**만 달리해 하나의 모델로 여러 NLP 작업을 수행한다는 점이 핵심.

2) 사전학습(비지도) 포인트

- **Denoising** 형태: 입력 일부 구간을 마스킹하여 복원
- **Sentinel Tokens** 사용: `<extra_id_0>`, `<extra_id_1>` ... (최대 100개 기본)
 - 입력: 원문 + sentinel
 - 출력: sentinel에 해당하는 복원 텍스트

3) T5 실습

- 개요: 텍스트→텍스트(encoder-decoder). 번역/요약/문장 유사도/문법성 등 **프롬프트 기반**으로 하나의 인터페이스로 처리.
- 프리트레인 스킴: **Sentinel 토큰**(`<extra_id_0>`, `<extra_id_1>` ...)을 써서 스펠 마스킹.
- 데이터: `argilla/news-summary` (test split에서 5,000 샘플 추출), 입력 앞에 **"summarize: "** 프리픽스.
- 전처리: `T5Tokenizer` 로 `max_length=128`, `pad_to_max_length=True`, `truncation=True` .
- 학습: **teacher forcing**
`decoder_input_ids = target[:, :-1]`, `labels = target[:, 1:]`, 패딩 위치 `-100` 으로 **loss ignore**.
손실 감소(예: Val Loss 3.32 → 2.76).
- 생성: `generate(max_length=128, num_beams=3, repetition_penalty=2.5, length_penalty=1.0, early_stopping=True)`
→ 생성 요약이 정답 요약과 **유사하지만 완전 동일하진 않음**(데이터·하이퍼파라미터 추가 튜닝 여지).

데이터셋 로드 (뉴스 요약)

```
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
```

```

df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["text"] = "summarize: " + df["text"]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])

train, valid, test = np.split(
    df.sample(frac=1, random_state=42),
    [int(0.6*len(df)), int(0.8*len(df))]
)

```

- **text**: 뉴스 본문 앞에 "**summarize:**" 접두어 부착
- **prediction**: 참조 요약문(정답)

전처리(토큰나이저 & 텐서셋)

```

import torch
from transformers import T5Tokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer, device):
    source = tokenizer(
        text=data.text.tolist(),
        padding="max_length",
        max_length=128,
        pad_to_max_length=True,
        truncation=True,
        return_tensors="pt"
    )
    target = tokenizer(
        text=data.prediction.tolist(),
        padding="max_length",
        max_length=128,
        pad_to_max_length=True,
        truncation=True,
        return_tensors="pt"
    )

```



```

source_ids = source["input_ids"].squeeze().to(device)
source_mask = source["attention_mask"].squeeze().to(device)
target_ids = target["input_ids"].squeeze().to(device)
target_mask = target["attention_mask"].squeeze().to(device)
return TensorDataset(source_ids, source_mask, target_ids, target_mask)

```

```

epochs = 3
batch_size = 8
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = T5Tokenizer.from_pretrained("t5-small")

```

- 토큰 인덱스 **21603, 10** 반복 = 앞에 붙인 **"summarize:"** 토큰들이라는 의미
- `pad_to_max_length=True`, `max_length=128` 로 패딩/자르기 일관화

모델 선언 & 구조

```

from torch import optim
from transformers import T5ForConditionalGeneration

model = T5ForConditionalGeneration.from_pretrained("t5-small").to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)

```

구조

- `shared` 임베딩: 인코더/디코더가 같은 임베딩 가중치 공유
 - `encoder.block` : 0~5
 - `decoder.block` : 0~5
 - `final_layer_norm`, `dropout`, `lm_head`
- 가중치 공유로 파라미터 수 감소 + 일반화 향상

T5 학습 루프(teacher forcing & pad ignore)

```

from transformers import T5ForConditionalGeneration
from torch import optim

model = T5ForConditionalGeneration.from_pretrained("t5-small").to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)

# batch에서
decoder_input_ids = target_ids[:, :-1].contiguous()
labels = target_ids[:, 1:].clone().detach()
labels[target_ids[:, 1:] == tok.pad_token_id] = -100 # ignore

outputs = model(
    input_ids=source_ids, attention_mask=source_mask,
    decoder_input_ids=decoder_input_ids, labels=labels
)
loss = outputs.loss
loss.backward()
optimizer.step()

```

T5 생성(평가용)

```

model.eval()
with torch.no_grad():
    gen_ids = model.generate(
        input_ids=source_ids, attention_mask=source_mask,
        max_length=128, num_beams=3,
        repetition_penalty=2.5, length_penalty=1.0,
        early_stopping=True
    )
    pred = tok.decode(gen_ids[0], skip_special_tokens=True,
                      clean_up_tokenization_spaces=True)
    ref = tok.decode(target_ids[0], skip_special_tokens=True,
                    clean_up_tokenization_spaces=True)

```

- 학습/검증 손실 (출력 결과)

- Epoch 1: **Train Loss 4.3135, Val Loss 3.3195** → Saved the model weights
- Epoch 2: **Train Loss 3.4098, Val Loss 2.9162** → Saved the model weights
- Epoch 3: **Train Loss 3.1287, Val Loss 2.7619** → Saved the model weights

- 생성 예시 (Generated vs Actual)

1. Generated: *Clinton leads Trump by 4 percentage points in four-... race for Nov. 8 election.*

Actual: *Clinton leads Trump by 4 points in Washington Post: ABC News poll*

2. Generated: *U.S. senators sharpen potential line of attack against Gorsuch's nomination to Supreme Court*

Actual: *Democrats question independence of Trump Supreme Court nominee*

3. Generated: *Saudi-led coalition pushed Riyadh to allow greater access for humanitarian aid...*

Actual: *In push for Yemen aid, U.S. warned Saudis of threats in Congress*

- 생성된 요약이 실제 요약과 크게 어긋나지 않고 유사한 핵심을 담음.
- 다만 완전 일치하진 않음 → 더 정확한 요약을 위해 학습 데이터 확대와 하이퍼 파라미터 튜닝이 유효.

용어 정리

- KoELECTRA 모델군

- 영어: `google/electra-small` (14M), `base` (110M), `large` (330M)
- 한국어: `monologg/koelectra-small-v3` (14M), `koelectra-base-v3` (110M)
- 다운스트림 분류엔 보통 **Discriminator**(`-discriminator`)를 사용.

- T5 특징

- 텍스트-투-텍스트 통일 인터페이스(입·출력 모두 시퀀스).

- **Sentinel 토큰**(`<extra_id*>`) 기반 **스팬 마스킹** 사전학습.
- 프롬프트 예시: `"translate English to German: ..."` , `"summarize: ..."` .
- **학습 포인트**
 - **Teacher forcing**: `decoder_input_ids = y[:, :-1]` , `labels = y[:, 1:]` .
 - **Loss ignore index**: 패딩 토큰을 **100** 으로 치환하여 손실에서 무시.
- **생성 하이퍼파라미터**
 - `num_beams` : 빔 서치 폭.
 - `repetition_penalty` : 중복 억제(↑ 더 적게 반복).
 - `length_penalty` : 길이 보상(↑ 더 길게).
 - `early_stopping` : EOS가 나오면 조기 종료.
 - `skip_special_tokens` , `clean_up_tokenization_spaces` : 디코딩 후 특수토큰/공백 정리.

실습 순서 정리

1. 토크나이저/모델 선택

- KoELECTRA: `"monologg/koelectra-base-v3-discriminator"`
- T5: `"t5-small"`

2. 데이터 준비

- `argilla/news-summary` 에서 5,000 샘플 추출 → 입력 앞에 `"summarize: "` .
- train/valid/test = **6:2:2** 분할.

3. 전처리 세팅(공통)

- `max_length=128` , `pad_to_max_length=True` , `truncation=True` .

4. KoELECTRA 분류 학습

- 옵티마이저 `AdamW(lr=1e-5, eps=1e-8)` .
- 출력: Val Acc 확인 → 기대치 **~0.87**.

5. T5 요약 학습

- Teacher forcing 구성(`decoder_input_ids` , `labels(-100)`).
- 에폭 진행하며 Train/Val Loss 로그로 감소 확인.

6. 생성 평가

- `generate()` 파라미터: (128, beam=3, rep=2.5, len=1.0, early_stopping=True).
- 생성 요약 vs 정답 요약 **텍스트로 직접 비교.**

7. 개선 포인트

- 더 큰 데이터, 에폭↑, 빔/패널티 튜닝으로 품질 향상 여지.